

Changes between C++14 and C++17 DIS

Abstract

This document enumerates all the major changes that have been applied to the C++ working draft since the publication of C++14, up to the publication of the C++17 DIS (N4660). Major changes are those that were added in the form of a dedicated paper, excluding those papers that are large issue resolutions. No issue resolutions from either CWG or LWG issues lists (“defect reports”) are included.

Contents

1. [Removed or deprecated features](#)
2. [New core language features with global applicability](#)
3. [New core language features with local applicability](#)
4. [New library features](#)
5. [Modifications to existing features](#)
6. [Miscellaneous](#)
7. [Unlisted papers](#)
8. [Assorted snippets demonstrating C++17](#)

Removed or deprecated features

Document	Summary	Examples, notes
N4086	Remove trigraphs	The sequence <code>??!</code> no longer means <code> </code> . Implementations may offer trigraph-like features as part of their input encoding.
P0001R1	Remove <code>register</code>	The <code>register</code> keyword remains reserved, but it no longer has any semantics.
P0002R1	Remove <code>++</code> for <code>bool</code>	Increment (<code>++</code>) prefix and postfix expressions are no longer valid for operands of type <code>bool</code> .
P0003R5	Remove <code>throw(A, B, C)</code>	Dynamic exception specifications of the form <code>throw(A, B, C)</code> are no longer valid. Only <code>throw()</code> remains as a synonym for <code>noexcept(true)</code> . Note the change in termination semantics.
P0386R2	Deprecate redeclaration of static constexpr class members	Given <code>struct X { static constexpr int n = 10; };, int X::n;</code> is no longer a definition, but instead a redundant redeclaration, which is deprecated. The member <code>X::n</code> is implicitly inline (see below).
N4190	Remove <code>auto_ptr</code> , <code>random_shuffle</code> , old parts of <code><functional></code>	Features that have been deprecated since C++11 and replaced with superior components are no longer included. Their names remain reserved, and implementations may choose to continue to ship the features.
P0004R1	Remove deprecated iostream aliases	Same as above
P0302R1	Remove allocator support from <code>function</code>	The polymorphic function wrapper <code>function</code> no longer has constructors that accept an allocator. Allocator support for type-

erasing, copyable types is difficult, and possibly not implementable efficiently.

P0063R3 (see below)	Deprecate C library headers	The following headers of the “C library” (this is the term for a part of the C++ standard library, <i>not</i> a part of the C standard!) are now deprecated: <code><ccomplex></code> , <code><cstdalign></code> , <code><cstdbool></code> , <code><ctgmath></code> . Note that the header <code><ciso646></code> is not deprecated.
P0174R2	Deprecate old library parts	These library components are now deprecated: <code>allocator<void></code> , <code>raw_storage_iterator</code> , <code>get_temporary_buffer</code> , <code>is_literal_type</code> , <code>std::iterator</code> .
P0618R0	Deprecate <code><codecvt></code>	The entire header <code><codecvt></code> (which does <i>not</i> contain the class <code>codecvt!</code>) is deprecated, as are the utilities <code>wstring_convert</code> and <code>wbuffer_convert</code> . These features are hard to use correctly, and there are doubts whether they are even specified correctly. Users should use dedicated text-processing libraries instead.
P0371R1	Deprecate <code>memory_order_consume</code> temporarily	The current semantics of “consume” ordering have been found inadequate, and the ordering needs to be redefined. While this work is in progress, hopefully ready for the next revision of C++, users are encouraged to not use this ordering and instead use “acquire” ordering, so as to not be exposed to a breaking change in the future.
P0521R0	Deprecate <code>shared_ptr::unique</code>	This member function suggests behaviour that is not actually provided.
P0604R0	Deprecate <code>result_of</code>	Use the new trait <code>invoke_result</code> instead.

New core language features with global applicability

These are features that may happen to you without your knowledge or consent.

Document	Summary	Examples, notes
P0012R1	Exception specification as part of the type system	The exception specification of a function is now part of the function’s type: <code>void f() noexcept(true)</code> ; and <code>void f() noexcept(false)</code> ; are functions of two distinct types. Function pointers are convertible in the sensible direction. (But the two functions <code>f</code> may not form an overload set.) This change strengthens the type system, e.g. by allowing APIs to require non-throwing callbacks.
P0135R1	Guaranteed copy elision	The meaning of <i>prvalue</i> and <i>glvalue</i> has been revised, <i>prvalues</i> are no longer objects, but merely “initialization”. Functions returning <i>prvalues</i> no longer copy objects (“mandatory copy elision”), and there is a new <i>prvalue-to-glvalue</i> conversion called <i>temporary materialization conversion</i> . This change means that copy elision is now guaranteed, and even applies to types that are not copyable or movable. This allows you to define functions that return such types.
P0035R4	Dynamic allocation of over-aligned types	Dynamic allocation (<code>operator new</code>) may now support over-aligned types, and a new overload of the operator takes an alignment parameter. It is still up to the implementation to choose which alignments to support.
P0145R3	Stricter order of expression evaluation	The order of evaluation of certain subexpressions has been specified more than it used to be. An important particular aspect of this change is that function arguments are now evaluated in an indeterminate order (i.e. no interleaving), which was previously merely unspecified. Note that the evaluation order for overloaded operators depends on how they are

invoked: when invoked using operator syntax, the order is the same as for the built-in operator, but when invoked using function call syntax, the order is the same as for ordinary function calls (i.e. indeterminate).

New core language features with local applicability

These are features where you would know if you were using them.

Document	Summary	Examples, notes
N4267	A <code>u8</code> character literal	A character literal prefix <code>u8</code> creates a character that is a valid Unicode code point that takes one code unit of UTF-8, i.e. an ASCII value: <code>u8'x'</code>
P0245R1	Hexadecimal floating point literals	Floating point literals with hexadecimal base and decimal exponent: <code>0xC.68p+2</code> , <code>0x1.P-126</code> . C has supported this syntax since C99, and <code>printf</code> supports it via <code>%a</code> .
N4295 , P0036R0	Fold expressions	A convenient syntax for applying a binary operator iteratively to the elements of a parameter pack: <code>template <typename ...Args> auto f(Args ...args) { return (0 + ... + args); }</code>
P0127R2	<code>template <auto></code>	A non-type template parameter may now be declared with placeholder type <code>auto</code> . Examples: <code>template <auto X> struct constant { static constexpr auto value = X; };</code> <code>Delegate<&MyClass::some_function></code>
P0091R3 , P0433R2 , P0512R0 , P0620R0	Class template argument deduction	The template arguments of a class template may now be deduced from a constructor. For example, <code>pair p(1, 'x');</code> defines <code>p</code> as <code>pair<int, char></code> (this is not an HTML error, the template arguments were omitted deliberately). The implicit deduction is complemented by a system of explicit deduction guides which allow authors to customise how the deduction happens, or forbid it.
P0292R2	Constexpr <code>if</code>	In a template specialization, the arms of the new <code>if constexpr</code> (<i>condition</i>) statement are only instantiated if the condition (which must be a constant expression) has the appropriate value.
P0305R1	Selection statements with initializer	The selection statements <code>if</code> and <code>switch</code> gain a new, optional initializer part: <code>if (auto it = m.find(key); it != m.end()) return it->second;</code>
P0170R1	Constexpr lambdas	Lambda expressions may now be constant expressions: <code>auto add = [] (int a, int b) constexpr { return a + b; }; int arr[add(1, 2)];</code>
P0018R3	Lambda capture of <code>*this</code>	Before: <code>[self = *this]{ self.f(); }</code> Now: <code>[*this]{ f(); }</code>
P0386R2	Inline variables	In a header file: <code>inline int n = 10;</code> All definitions refer to the same entity. Implied for static constexpr class data members.
P0217R3 , P0615R0	Structured bindings	<code>auto [it, ins] = m.try_emplace(key, a1, a2, a3);</code> Decomposes arrays, all-members-public classes, and user-defined types that follow a <code>get<N></code> protocol like <code>pair</code> and <code>tuple</code> .
P0061R1	<code>__has_include</code>	A preprocessor operator to check whether an inclusion is possible.
P0188R1 P0189R1	Attribute <code>[[fallthrough]]</code>	A new set of standardised attributes. The attributes formally have no required semantics, but implementations are encouraged to emit or

P0212R1	Attribute [[nodiscard]] Attribute [[maybe_unused]]	suppress the appropriate diagnostics (warnings).
P0137R1	launder	A language support tool (an “optimisation barrier”) to allow libraries to reuse storage and access that storage through an old pointer, which was previously not allowed. (This is an expert tool for implementers and not expected to show up in “normal” code.)
P0298R3	A byte type	A new type <code>byte</code> is defined in <code><cstdint></code> (not in <code><stdint.h></code> , and only in namespace <code>std</code> !) which has the layout of <code>unsigned char</code> , shares the aliasing allowances of the existing char types, and has bitwise operations defined.

New library features

Document	Summary	Examples, notes
P0226R1	Mathematical special functions	The contents of the former international standard ISO/IEC 29124:2010 (mathematical special functions) are now part of C++. The functions were added only to <code><cmath></code> , not to <code><math.h></code> , and are only available in namespace <code>std</code> .
P0218R0, P0219R1, P0317R1, P0392R0, P0430R2, P0492R2	Filesystem	The contents of the Filesystems Technical Specification are now part of C++. The filesystems library allows portable interaction with directories and directory-like structures (listing directory contents, moving files, etc.). It is largely modelled on POSIX, but flexible enough to be implementable for a wide variety of systems.
P0024R2, P0336R1, P0394R4, P0452R1, P0467R2, P0502R0, P0518R1, P0523R1, P0574R1, P0623R0	Parallelism	The contents of the Parallelism Technical Specification are now part of C++. This adds new overloads, taking an additional <i>execution policy</i> argument, to many algorithms, as well as entirely new algorithms (see below). Three execution policies are supported, which respectively provide sequential, parallel, and vectorized execution.
P0024R2	New algorithms	The Parallelism Technical Specification adds several new algorithms to the standard library. They are motivated by their potential for efficient parallel execution, but are available in the usual simple form as well: <code>for_each_n</code> , <code>reduce</code> , <code>transform_reduce</code> , <code>exclusive_scan</code> , <code>inclusive_scan</code> , <code>transform_exclusive_scan</code> , <code>transform_inclusive_scan</code> . Note that <code>reduce</code> looks similar to the existing <code>accumulate</code> , but does not guarantee any particular order of operations.
P02202, P0254R2, P0403R1	New type: <code>string_view</code> (and <code>basic_string_view</code>)	The new <code>string_view</code> class is the preferred interface vocabulary type for APIs that need to view a string without wanting to take ownership or to modify it. It

is constructible from char pointers, but all other classes that are string-like should offer conversions to `string_view`.

P02202, P0032R3, P0504R0	New type: <code>any</code>	The type <code>any</code> type-erases copyable objects. There are essentially three things you can do with an <code>any</code> : 1. put a value of type <code>T</code> into it. 2. Make a copy of it. 3. Ask it whether it contains a value of type <code>U</code> and get that value out, which succeeds if and only if <code>U</code> is <code>T</code> .
P0088R3, P0393R3, P0032R3, P0504R0, P0510R0	New class template: <code>variant</code>	A variant models a disjoint union (or discriminated union). A value of <code>variant<A, B, C></code> contains <i>one of</i> an <code>A</code> , a <code>B</code> , or a <code>C</code> at any one time.
P02202, P0307R2, P0032R3, P0504R0	New class template: <code>optional</code>	An optional value. A <code>optional<T></code> represents either a <code>T</code> value, or no value (which is signified by the tag type <code>nullopt_t</code>). In some respects this can be thought of as equivalent to <code>variant<nullopt_t, T></code> , but with a purpose-built interface.
N4169	<code>invoke</code>	A facility to uniformly invoke callable entities. This allows users to write libraries with the same behaviour as the standard's magic <i>INVOKE</i> rule.
P0077R2, P0604R0	<code>is_invocable</code> , <code>is_invocable_r</code> , <code>invoke_result</code>	Traits to reason about invocability and invocation results.
P0067R5	Elementary string conversions	Functions <code>to_chars</code> , <code>from_chars</code> that produce or parse string representations of numbers. These are intended to form an efficient, low-level basis for a replacement for <code>printf</code> and iostream formatted operations. They follow idiomatic C++ algorithm style.
N3911	Alias template <code>void_t</code>	<pre>template <class...> using void_t = void;</pre> Surprisingly useful for metaprogramming, to simplify use of SFINAE.
N4389	Alias template <code>bool_constant</code>	<pre>template <bool B> using bool_constant = integral_constant<bool, B></pre>
P0013R1	Logical operation metafunctions	Variadic metafunctions <code>conjunction</code> , <code>disjunction</code> , and <code>negation</code> for metaprogramming. These traits short-circuit in the metaprogramming sense: template specializations that are not required to determine the result are not instantiated.
P0185R1	Traits for SFINAE-friendly <code>swap</code>	New traits <code>is_{,nothrow_}swappable</code> , <code>is_{,nothrow_}swappable_with</code> .
LWG 2911	Trait <code>is_aggregate</code>	Whether a type is an aggregate. Useful for example to tell whether a generic type should be list- or non-list-initialized.
P0258R2	Trait <code>has_unique_object_representations</code>	This trait may be used to reason about whether certain value-based operations like comparison and hashing can be replaced with representation-based operations (e.g. <code>memcmp</code>).
P0007R1	<code>as_const</code>	Given an lvalue <code>x</code> , <code>as_const(x)</code> returns the const-

qualified version. Does not bind to rvalues.

N4280	Non-member <code>size</code> , <code>data</code> , <code>empty</code>	The additional functions complement the existing free functions <code>begin</code> , <code>end</code> etc. to access containers and arrays in a uniform fashion. Note that unlike <code>begin/end</code> , the new functions are <i>not</i> customisation points for anything and are only provided for convenience.
P0025R0	<code>clamp</code>	<code>clamp(x, low, high)</code> returns either <code>x</code> if <code>x</code> is within the interval <code>[low, high]</code> , or the nearest bound otherwise.
P0295R0	<code>gcd</code> and <code>lcm</code>	Number-theoretic functions to compute the greatest common divisor and least common multiple of two integers.
N4508	Class <code>shared_mutex</code>	A reader-writer mutex, which can be locked in either shared or exclusive mode.
P0154R1	Interference sizes	Two new implementation-defined constants <code>hardware_construction_interference_size</code> that effectively allow the platform to document its cache line sizes so that users can avoid false sharing (destructive interference) and improve locality (constructive interference). Two separate constants are defined to support heterogeneous architectures.
P0220R1	Tuple <code>apply</code>	Invokes a callable with arguments extracted from a given tuple.
P0209R2	Construction from tuples	A new function template <code>make_from_tuple</code> that initializes a value of type <code>T</code> from the elements of a given tuple. It is like <code>apply</code> applied to a constructor.
P0005R4, P0358R1	Universal negator <code>not_fn</code>	A call wrapper that negates its wrapped callable. This works with callables of any arity and replaces the old <code>not1</code> and <code>not2</code> wrappers.
P0220R1	Memory resources	A new set of components comprised of a <i>memory resource</i> base class for dynamically selectable memory providers, as well as three concrete implementations (<code>synchronized_pool_resource</code> , <code>unsynchronized_pool_resource</code> , <code>monotonic_buffer_resource</code>). See next item for use cases.
P0220R1, P0337R0	A polymorphic allocator	An allocator that uses a memory resource, which can be changed at runtime and is not part of the allocator type. Also contains convenience type aliases like <code>std::pmr::vector<T> = std::vector<T, polymorphic_allocator<T>></code> .
P0220R1, P0253R1	Searcher functors	Substring searcher functors implementing the Boyer-Moore and Boyer-Moore-Horspool algorithms, and a <code>search</code> algorithm using those functors.

Modifications to existing features

Document	Summary	Examples, notes
N3928	Single-argument <code>static_assert</code>	The <code>static_assert</code> declaration no longer requires a second argument: <code>static_assert(N > 0);</code>
N4230	Nested namespace declarations	<code>namespace foo::bar { /* ... */ }</code>
N4051	Allow <code>typename</code> in template template parameters	<code>template <template <typename> typename Tmpl> struct X;</code> Previously, template template parameters were required to use the keyword <code>class</code> .
P0184R0	Range-based <code>for</code> takes separate begin/end types	The rewrite rule for <code>for (decl : expr)</code> now says <code>auto __begin = begin-expr; auto __end = end-expr;</code> , as opposed to <code>auto __begin = begin-expr, __end = end-expr;</code> before. This prepares the range-based <code>for</code> statement for the new Ranges (work in progress).
P0195R2	Pack expansion in <i>using-declarations</i>	<code>template <typename ...Args> struct X : Args... { using Args::f...; };</code>
P0138R2	Construction for values of fixed enums	A variable of a fixed enumeration <code>E</code> can now be defined with <code>E e { 5 };</code> and no longer requires the the more cumbersome <code>E e { E(5) };</code> .
N4259	<code>uncaught_exceptions()</code>	The function <code>uncaught_exception</code> is deprecated, the new function <code>uncaught_exceptions</code> returns a count rather than a boolean. The previous feature was effectively unusable; N4152 explains the details.
N4266	Attributes in namespaces and enumerators	Namespaces and enumerators can now be annotated with attributes. This allows, for example, to deprecate namespaces or enumerators.
P0028R4	Attribute namespaces without repetition	This simplifies the use of attribute namespace qualifications when a namespace is used repeatedly.
N4279	Improved insertion for unique-key maps	<code>m.try_emplace(key, arg1, arg2, arg3)</code> does nothing if <code>key</code> already exists in the map, and otherwise inserts a new element constructed from the arguments. This interface guarantees that even if the arguments are bound to rvalue references, they are not moved from if the insertion does not take place.
P0084R2	Return type of <code>emplace</code>	Sequence containers whose <code>emplace{, _front, _back}</code> member function templates used to return <code>void</code> now return a reference to the newly inserted element. (Associative containers are not affected, since their insertion functions have always returned iterators to the relevant element.)
P0083R3 , P0508R0	Splicing maps and sets	A new mechanism, <i>node handles</i> , has been added to the container library that allows transplanting elements between different map/set objects without touching the contained object. Moreover, this technique enables mutable access to key values of extracted nodes.
P0272R1	Non-const <code>string::data</code>	There is now a non-const overload of <code>basic_string::data</code> that returns a mutable pointer. Moreover, C++17 allows writing to the null terminator, provided that the value zero is written. This makes the string classes a bit more convenient to use with C-style interfaces.
P0156R0 , P0157R0	A variadic version of <code>scoped_lock</code>	A new, variadic class template <code>scoped_lock<Args...></code> that

P0156R2	<code>lock_guard</code> called <code>scoped_lock</code>	locks multiple lockable objects at once (using the same algorithm as <code>lock</code>) and releases them in the destructor. Initially it was suggested to simply change the definition of <code>lock_guard</code> to become variadic, but this was discovered to be a breaking change, and so instead we now have a new class template <code>scoped_lock</code> that is strictly superior to the old <code>lock_guard</code> and should be used instead.
P0006R0	Variable templates for traits	For every standard type trait <code>foo</code> with a single, static member constant <code>foo<Args...>::value</code> , there is now a variable template <code>foo_v<Args...></code> .
P0152R1	<code>atomic::is_always_lock_free</code>	A new static member constant <code>is_always_lock_free</code> that documents whether the operations of a given atomic type are always lock-free. The existing non-static member function <code>is_lock_free</code> may give different answers for different values of the atomic type.
P0220R1, P0414R2	<code>shared_ptr</code> for arrays	The class template <code>shared_ptr</code> now supports C-style arrays by passing <code>T[]</code> or <code>T[N]</code> as the template argument, and the constructor from a raw pointer will install an appropriate array deleter.
P0163R0	<code>shared_ptr::weak_type</code>	The class <code>shared_ptr<T></code> now has a member type <code>weak_type</code> which is <code>weak_ptr<T></code> . This allows generic code to name the corresponding weak pointer type without having to destructure the shared pointer type.
P0030R1	Three-dimensional hypotenuse	The three-dimensional hypotenuse <code>hypot(x, y, z)</code> is added as an additional set of overloads to <code><cmath></code> (but not to <code><math.h></code> and only to namespace <code>std</code>).
P0040R3	Further uninitialized algorithms	Additional algorithms to create objects in uninitialized memory and to destroy objects. Separate versions for default- and value-initialization are included.
N4510	Incomplete type support for allocators	This change relaxes the requirements on allocators to have complete value types, and allows, for example, recursive structures like: <code>struct X { std::vector<X> data; };</code>
P0092R1, P0505R0	Changes to <code><chrono></code>	Adds floor, ceiling, division and rounding for time points; makes most member functions constexpr.
P0426R1	Constexpr for <code>char_traits</code>	All specializations required for <code>char_traits</code> now have constexpr member functions <code>length</code> , <code>compare</code> , <code>find</code> and <code>assign</code> , allowing string views to be more widely used in constant expressions.
N4387	Improving <code>pair</code> and <code>tuple</code>	This change makes the constructors of <code>pair</code> and <code>tuple</code> as explicit as the corresponding element type constructors.
P0435R1, P0548R1	Changes to <code>common_type</code>	Because it's not a new standard if we didn't make changes to <code>common_type</code> ...

Miscellaneous

Document	Summary	Examples, notes
P0063R3	C++ refers to C11	The C++ standard now refers normatively to C11 (ISO/IEC 9899:2011)

as “The C Standard”. Not only does ISO require that references to other international standards refer to the latest published version, and not to a historic version, but this also gives us access to [aligned_alloc](#), which is useful for the improvements to our dynamic memory management.

P0180R2	Reserved namespaces	All top-level namespaces of the form <code>stdX</code> , where <i>X</i> is a sequence of digits, are reserved.
P0175R1	C library synopses	A purely editorial change: all headers of the “C library” part of the standard library are now presented as complete synopses in the C++ standard document, rather than as just lists of names. This makes the changes in semantics from C easier to appreciate (e.g. additional overloads, overloads on language linkage).
N4262 P0134R0 P0391R0 N4284	Term “forwarding reference” Term “default member initializer” Term “templated entity” Term “contiguous iterator”	These changes have no normative impact, but they establish official terminology for concepts that have so far only <i>emerged</i> from the language rules. Having precise and well-known terms simplifies talking about C++ and simplifies the specification.
P0346R1	Change “random number generator” to “random bit generator”	Similarly, this change has no normative impact, but clarifies the design and intended use of this aspect of the <code><random></code> facilities.

Unlisted papers

The following papers were moved at committee meetings, but their contents are too specific to call out as separate features: [N3922](#), [N4089](#), [N4258](#), [N4261](#), [N4268](#), [N4277](#), [N4285](#), [P0017R1](#), [P0031R0](#), [P0033R1](#), [P0074R0](#), [P0136R1](#), [P0250R3](#), [P0270R3](#), [P0283R2](#), [P0296R2](#), [P0418R2](#), [P0503R0](#), [P0509R1](#), [P0513R0](#), [P0516R0](#), [P0517R0](#), [P0558R1](#), [P0599R1](#), [P0607R0](#), [P0612R0](#)

The following papers contain issues that have been accepted as defect reports. CWG issues are handled by [N4192](#), [N4457](#), [P0164R0](#), [P0167R2](#), [P0263R1](#), [P0384R0](#), [P0398R0](#), [P0490R0](#), [P0507R0](#), [P0519R0](#), [P0520R0](#), [P0522R0](#), [P0575R1](#), [P0576R1](#), [P0613R0](#), [P0622R0](#). LWG issues are handled by [N4245](#), [N4366](#), [N4383](#), [N4525](#), [P0165R0](#), [P0165R1](#), [P0165R2](#), [P0165R2](#), [P0165R3](#), [P0165R4](#), [P0304R1](#), [P0397R0](#), [P0610R0](#), [P0625R0](#). Only specific issues may have been selected from each paper; the meeting minutes contain the details.

Assorted snippets demonstrating C++17

```
std::unordered_map<std::string, std::unique_ptr<Foo>> items;
std::vector<std::unique_ptr<Foo>> standby;

// If there is currently no item 'id', installs 'foo' as item 'id'.
// Otherwise stores 'foo' for later use and puts it on standby.

// Before C++17

void f(std::string id, std::unique_ptr<Foo> foo) {
    auto it = items.find(id);
    if (it == items.end()) {
        auto p = items.emplace(std::move(id), std::move(foo));
        p.first->second->launch();
    } else {
```

```

    standby.push_back(std::move(foo));
    standby.back()->wait_for_notification();
}

// Notes:
// * Variable 'id' can no longer be used (moved-from); or...
// * ...would need to use parameter 'const string& id' and force copying.
// * Map lookup performed twice. Ordered map could use lower_bound + hint, but unordered
map cannot.
// * (Cannot emplace unconditionally, because it might destroy *foo.)
}

// With C++17

void f(std::string_view id, std::unique_ptr<Foo> foo) {
    if (auto [pos, inserted] = items.try_emplace(id, std::move(foo)); inserted) {
        pos->second->launch();
    } else {
        standby.emplace_back(std::move(foo))->wait_for_notification();
    }
}

```

The next snippet illustrates the utility of `template <auto>` on the example of a class template which delegates a free function call to a member function bound to a class instance, and the member function is part of the delegate *type*.

```

// Before C++17

template <typename T, int (T::* MF)(int, int)> // two params: one type, one non-type
struct Delegate { /* ... */ };

int n = Delegate<MyComplexClass, &MyComplexClass::an_important_function>(&obj)(10, 20);

// With C++17

template <auto> struct Delegate; // one (non-type) param
template <typename T, int (T::* MF)(int, int)>
struct Delegate<MF> { /* ... */ }; // implement as before, but as partial
specialization

int n = Delegate<&MyComplexClass::an_important_function>(&obj)(10, 20);

```

The next snippet shows the utility of fold expressions in generic code.

```

// Call f(n) for all f in the pack.
template <typename ...F>
void ApplyAll(int n, const F&... f) {
    (f(n), ...); // unary fold (over the comma operator)
}

// Compute f(a, b) for each f in the pack and return the sum.
template <typename ...F>
int ApplyAndSum(int a, int b, const F&... f) {
    return (f(a, b) + ... + 0); // binary fold
}

```

The next snippet shows array support for shared pointers.

// Before C++17

```
std::shared_ptr<char> p(new char[N], std::default_delete<char[]>()); // would be wrong  
without the deleter
```

// With C++17

```
std::shared_ptr<char[]> p(new char[N]); // deleter uses "delete[]"
```